Assignment - 2

Module 2 – Introduction to Programming

1. Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today ?

⇨ **History and Evolution of C Programming :-**

- C Programming language was developed by Dennis Ritchie In the early 1970s at Bell Laboratories.
- It was created to develop the UNIX Operating system.
- C evolved from earlier languages like B and BCPL.
- In 1973, UNIX was rewritten in C , which made C very popular.
- The language was later standardize as ANSI C (1989) and improved versions like C99 and C11.

⇨ **Importance of C Programming :-**

- C is a fast and efficient language.
- It provides low-level access to memory using pointers.
- Used for system programming, operating system and embedded system.
- Many modern operating systems like Windows, Linux and macOS use C.

⇨ **Why C Is Still Used Today :-**

- C Program runs faster than many high-level languages.
- It is portable, meaning it works on many platforms.
- A large amount of existing software is written in C.
- C is the base language for many other languages like C++, Java and Python.
- It helps programmers understand how computers work internally.

2. Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development. - LAB EXERCISE

⇨ Below explanation :-

⇨ **Operating Systems :-**

- C is widely used to develop operating system.
- Major OS Component Like Linux, Windows and macOS are written largely in C.
- C is fast and efficient.
- It provides direct access to hardware and memory.
- Example :- Linux Kernel is mostly written in C.

⇨ **Embedded Systems :-**

- C is the most popular language for embedded systems.
- Used in devices like :-
  - ✓ Washing Machine
  - ✓ Microwave Ovens
  - ✓ Medical Devices
  - ✓ Cars
- C uses less memory
- Works well on low-power hardware
- Examples:- Microcontrollers like Arduino and PIC use C Programs.

⇨ **Game Development :-**

- C is used in Game Engine and performance-critical parts of games.
- C offers high performance.
- Allows efficient graphics and memory handling.
- Examples :- Game Engine like Unity (core parts) and unreal engine (low-level systems) use C/C++.

3. Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

⇨ **Steps to Install a C Compiler (GCC) :-**

- **Download GCC :-**
  - ✓ Windows :- Download MinGW or TDM-GCC
- **Install the compiler :-**
  - ✓ Run the installer and follow default steps
- **Set Path (Windows Only) :-**

- ✓ Add the bin folder of GCC to Environment Variables -> Path

- **Verify Installation :-**
  - ✓ Open Command Prompt / Terminal
  - ✓ Type: GCC –version.

⇨ **Steps to Set Up Dev-C++ IDE :-**

- **Download Dev-C++ :-**
  - ✓ Download from the official Dev-C++ website.
- **Install Dev-C++ :-**
  - ✓ Run setup and finish Installation
- **Configure Compiler :-**
  - ✓ Dev-C++ usually detects GCC automatically
  - ✓ Go to Tools -> Compiler Options if needed.
- **Create First Program :-**
  - ✓ File -> New -> Source File
  - ✓ Write C code and save with .c extension.
- **Compile & Run :-**
  - ✓ Click Compile & Run (F11).

4. Install a C compiler on your system and configure the IDE. Write your first program to print "Hello, World!" and run it. - LAB EXERCISE

⇨ **Input :-**

areaofcircle.c × AreaofRect.c × calculator.c ×

```c
#include<stdio.h>

main(){
    printf("Hello World!!\n");
    printf("Welcome to our project");
}
```

Output :-

```
E:\C# Practices\TOPS LAB PRA
Hello World!!
Welcome to our project
--------------------------------
Process exited after 0.07365 seconds with return value 0
Press any key to continue . . .
```

5. Explain the basic structure of a C program, including headers, main function, comments, data types, and variables. Provide examples.

⇨ Basic structure of C program :-

- **Header Files :-**
  - ✓ Header files can contain built-in functions and definitions.
  - ✓ They are included using #include.
  - ✓ Example :- #include<stdio.h> is used for input/output functions like printf().
- **Main Function :-**
  - ✓ main() function where programs start execution.
  - ✓ It must return an int value.
  - ✓ Return 0; when program ran successfully.
  - ✓ Example :-

```
Int main(){
    return 0;
}
```

- **Comments :-**
  - ✓ Comments explain the code and ignored by compiler.
  - ✓ There are two types of comments :-
  - ✓ Single line of comments (//)
  - ✓ Multiple line of comments (/**/)
  - ✓ Example :-

```
//Hello World
/*12345
Hello guys!
```

*/

- **Data Types and Variables :-**
  - ✓ Variables can store data and must have a data type.
  - ✓ Common data types :-

    Int → Whole Number

    Float → Decimal Number

    Double → more Precise Decimals

    Char → Single character
  - ✓ Example :-

    int age = 14;

    float num = 23.4;

    double dl = 34.56;

    char ch = 'Y';

6. Write a C program that includes variables, constants, and comments. Declare and use different data types (int, char, float) and display their values. - LAB EXERCISE

⇨ **Input :-**

different.c

```c
and use different data types (int, char, float) and display their values.*/

#include<stdio.h>

int main(){
    //varaible declaration
    int a;
    char b;
    float c;

    //constant value declare
    const float z = 3.14;

    printf("Enter the Integer Value of a: ");
    scanf("%d",&a);

    printf("Enter the single character :");
    scanf(" %c",&b);

    printf("Enter the floating value :");
    scanf("%f",&c);

    //print the value
    printf("\nPosiitve Value is: %d",a);
    printf("\nSingle Character is: %c",b);
    printf("\nfloating point is: %.2f",c);
    printf("\nconstant value is: %.2f",z);
    return 0;
}
```

Output :-

```
E:\C# Practices\TOPS LAB PRA

Enter the Integer Value of a: 12
Enter the single character :Y
Enter the floating value :77.7

Posiitve Value is: 12
Single Character is: Y
floating point is: 77.70
constant value is: 3.14
--------------------------------
Process exited after 13.62 seconds with return value 0
Press any key to continue . . .
```

7. Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.

⇨ Here's Below the explanation of operator

⇨ **Arithmetic Operator :-**

Operator	Name	Description	Example
+	Addition	Add Two Operands.	5 + 2 = 7
-	Subtraction	Subtract right operands from left	5 – 2 = 3
*	Multiplication	Multiplies by Two Operands	5 * 2 = 10
/	Division	Divides left operands from right	5 / 2 = 2.5
%	Modules	Return the reminder of an integer division	5 % 2 = 0.1

⇨ **Relational Operator :-**

Operator	Name	Description	Example

==	Equal to	Check if two operands are equal.	5 == 2 evaluates to 0 (false)
!=	Not Equal to	Check if two operands are not equal	5 == 2 evaluates to 0 (false)
<	Less than	Checks if left operand is less than right	5 < 2 evaluates to 0 (false)
>	Greater than	Checks if left operand is greater than right	5 > 2 evaluates to 1 (true)
<=	Less than or Equal to	Checks if left operand is less than or equal to	5 <= 2 evaluates to 0 (false)
>=	Greater than or Equal to	Checks if left operand is greater than or equal to	5 >= 2 evaluates to 1 (true)

⇨ **Logical Operator :-**

Operator	Name	Description	Example
&&	Logical AND	Return 1 if both the conditions are true.	(5 > 2) && (2 < 5) evaluates to 1
||	Logical OR	If one condition are true than true otherwise false	Logical OR
!	Logical NOT	Reverses the logical state of an operand	(5 == 2) evaluates to 1

⇨ **Assignment Operator :-**

Operator	Name	Description	Example	Equivalent to
=	Assignment	Assign the value on right to the variable on the left	a = 5	a = 5
+=	Add and Assign	Adds right operand to the left operand and assigns the result to the left.	a+=2	a = a + 2
-=	Subtract and Assign	Subtracts right operand from the left operand and assigns the result to the left.	a-=2	a = a - 2
*=	Multiply and Assign	Multiplies right operand with the left operand and assigns the result to the left.	a*=2	a = a * 2
/=	Divide and Assign	Divides left operand by the right operand and assigns the result to the left.	a/=2	a = a / 2

%=	Modules and Assign	Takes modulus using two operands and assigns the result to the left operand.	a%=2	a = a % 2

⇨ **Increment/Decrement Operators :-**

Operator	Name	Description	Example
++	Increment	Increases the integer value by one.	++a (prefix) or a++ (postfix)
--	Decrement	Decrement the integer value by one.	--a (prefix) or a-- (postfix)

⇨ **Bitwise Operator :-**

Operator	Name	Description
&	Bitwise AND	Sets each bit to 1 if both bits are 1.
|	Bitwise OR	Sets each bit to 1 if either of the bits is 1.
^	Bitwise XOR	Sets each bit to 1 if only one of the bits is 1.
~	Bitwise Complement	Inverts all the bits of the operand.

<<	Left shift	Shifts the bits of the left operand to the left by the number of positions specified by the right operand.
>>	Right shift	Shifts the bits of the left operand to the right by the number of positions specified by the right operand.

⇨ **Conditional Operators :-**

Operator	Name	Syntax	Description	Example
?:	Conditional	condition ? expression1 : expression2	If the condition is true (1), expression1 is executed; otherwise, expression2 is executed.	Result = (marks >=60)? 'pass' : 'fail'

8. Write a C program to check if a number is even or odd using an if-else statement. Extend the program using a switch statement to display the month name based on the user's input (1 for January, 2 for February, etc.). - LAB EXERCISE

⇨ **Input :-**

different.c even-odd-month.c

```c
/*
Write a C program to check if a number is even or odd using an if-else
statement. Extend the program using a switch statement to display the month
name based on the user's input (1 for January, 2 for February, etc.).
*/

//Part 1 :- even or odd program
/*
even and odd formula :- num % 2  == 0
*/
#include<stdio.h>

int main()
{
    //variable declaration
    int num,month;

    printf("Enter the Number :");
    scanf("%d",&num);

    //if-else condition
    if(num%2==0)
    {
        printf("%d is Even Number",num);
    }else{
        printf("%d is Odd Number",num);
    }

    //part-2 :- month using switch statement
    printf("\n Enter the month number (1-12): ");
    scanf("%d",&month);

    switch(month){
        case 1:
            printf("Month : January");
        break;

        case 2:
            printf("Month : February");
        break;

        case 3:
            printf("Month : March");
        break;

        case 4:
            printf("Month : April");
        break;

        case 5:
            printf("Month : May");
        break;

        case 6:
            printf("Month : June");
        break;

        case 7:
            printf("Month : July");
        break;

        case 8:
            printf("Month : August");
        break;

        case 9:
            printf("Month : September");
        break;

        case 10:
            printf("Month : October");
        break;

        case 11:
            printf("Month : November");
        break;

        case 12:
            printf("Month : December");
        break;
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the Number :7
7 is Odd Number
 Enter the month number (1-12): 7
Month : July
--------------------------------
Process exited after 16.58 seconds with return value 0
Press any key to continue . . .
```

9. Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.

⇨ **Decision making statement :-**

➢ **If Statement :-**

✓ Executes a block of code only if the condition is true.

✓ Example :-

```
If(age>18){
        Printf("eligible for voting");
}
```

- **If-else Statement :-**
  - ✓ Executes one block if the condition is true.
  - ✓ Executes another block if the condition is false.
  - ✓ Example :-

```
If(age>=18){
    Printf("Eligible for voting");
}else{
    Printf("Not Eligible for voting");
}
```

- **Nested if-else :-**
  - ✓ An if-else statement inside another if-else
  - ✓ Used for check multiple condition.
  - ✓ Example :-

```
If(marks>=90){
    Printf("Grade A");
}else if(marks>=75){
    Printf("Grade B");
}else{
    Printf("Grade C");
}
```

- **Switch Statement :-**
  - ✓ Used when there are multiple fixed choices for a single variable.
  - ✓ Use Case, break and default.
  - ✓ Example :-

```
Switch(day){
    Case 1:
        Printf("Monday");
    Break;
    Case 2:
```

```
        Printf("Tuesday");
    Break;
    Default:
        Printf("Invalid Day");
    Break;
}
```

10. Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate.

⇨ Here's the below the difference:-

Feature	While loop	For loop	Do While loop
Condition Check	Checked before each iteration	Checked before each iteration	Checked after each iteration
Minimum Execution	0 times	0 times	At least one time
Loop Structure	Condition only	Initialization, Condition and Modification	Body-first and Condition Last
Risk of infinite loop	Medium	Low	Medium
Typical Syntax Style	While(condition)	For(Initialization; condition; modification)	Do { } while(condition)

- **Scenario and most appropriate loop :-**
  - ✓ While loop :-
  - ✓ Scenario :-
    - You don't know how many times the loop should run.
    - Monitoring the changing system state.
  - ✓ For loop :-
  - ✓ Scenario :-

- You are iterating fixed number of times.
- Traversing an array or list with an index.

✓ Do While loop :-

✓ Scenario :-

- You need to loop body to execute at least once.

11. Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while). - LAB EXERCISE

⇨ While loop :-

Input :-

while-loop.c

```c
/*
    2) while loop
*/

#include<stdio.h>

int main()
{
    int i = 1;

    while(i<=10){
        printf("\n i = %d",i);
        i++;
    }

    return 0;
}
```

Output:-

E:\TOPS Tech Documents\Mo

```
 i = 1
 i = 2
 i = 3
 i = 4
 i = 5
 i = 6
 i = 7
 i = 8
 i = 9
 i = 10
--------------------------------
Process exited after 0.05875 seconds with return value 0
Press any key to continue . . .
```

⇨ For loop :-

Input:-

while-loop.c for-loop.c

```c
//for loop

#include<stdio.h>

int main()
{
    //variable declaration
    int i;

    //using for loop
    for(i=1;i<=10;i++){
        printf("\n i=%d",i);
    }

    return 0;
}
```

Output:-

E:\TOPS Tech Documents\Mo

```
 i=1
 i=2
 i=3
 i=4
 i=5
 i=6
 i=7
 i=8
 i=9
 i=10
--------------------------------
Process exited after 0.05547 seconds with return value 0
Press any key to continue . . .
```

⇨ Do While loop :-

Input:-

```c
/*
    3) do..while loop
*/

#include<stdio.h>

int main()
{
    int i = 1;
    do{
        printf("\n i=%d",i);
        i++;
    }while(i<=10);

    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

 i=1
 i=2
 i=3
 i=4
 i=5
 i=6
 i=7
 i=8
 i=9
 i=10
--------------------------------
Process exited after 0.05547 seconds with return value 0
Press any key to continue . . .
```

12. Explain the use of break, continue, and goto statements in C. Provide examples of each.

⇨ Below the Explanation of each :-

- **Break Statement :-**
  - ✓ Used to terminate a loop or switch statement immediately.
  - ✓ Control goes to the statement after the loop or switch.
  - ❖ **Where to used :-** for, while, do..while and switch.
  - ❖ **Example :-**

    Input:-

```c
//break, continue and goto statement example

#include<stdio.h>
int main()
{
    int i;

    for(i=1;i<=5;i++){
        if(i==4){
            break;
        }
        printf("\ni=%d",i);
    }
    return 0;
}
```

Output:-

```
i=1
i=2
i=3
--------------------------------
Process exited after 0.1036 seconds with return value 0
Press any key to continue . . .
```

- **Continue Statement :-**
  - ✓ Used to skip current iteration of loop.
  - ✓ Loop continues with the next iteration.
    - ❖ **Where to used :-** for, while, do..while and switch.
    - ❖ **Example :-**

Input:-

```c
//break, continue and goto statement

#include<stdio.h>
int main()
{
    int i;

    for(i=1;i<=5;i++){
        if(i==4){
            continue;
        }
        printf("\ni=%d",i);
    }
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

i=1
i=2
i=3
i=5
--------------------------------
Process exited after 0.09702 seconds with return value 0
Press any key to continue . . .
```

- **Goto Statement :-**
  - ✓ Used to jump a specific label within a same function.
    - ❖ **Example :-**

Input:-

break-conti-task.c pattern-15.c patte

```c

#include<stdio.h>
int main()
{
    int num;

    printf("Enter the Number: ");
    scanf("%d",&num);

    if(num % 2 == 0){
        goto Even;
    }else{
        goto Odd;
    }

    Even:{
        printf("\n Number is Even");
        return;
    }

    Odd:{
        printf("\n Number is Odd");
        return;
    }
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the Number: 77

 Number is Odd
--------------------------------
Process exited after 1.503 seconds with return value 15
Press any key to continue . . .
```

13. Write a C program that uses the break statement to stop printing numbers when it reaches 5. Modify the program to skip printing the number 3 using the continue statement. - LAB EXERCISE

⇨ Below the Input and output :-

Input:-

break-con-goto.c

```c
/*
    Write a C program that uses the break statement to stop printing numbers
    when it reaches 5. Modify the program to skip printing the number 3 using the
    continue statement.
*/

#include<stdio.h>
int main()
{
    int i;

    for(i=1;i<=5;i++){
        if(i==5){
            break;
        }else if(i==3){
            continue;
        }
        printf("\ni=%d",i);
    }
    return 0;
}
```

Output:-

Break Statement Output:-

```
E:\TOPS Tech Documents\Mo

i=1
i=2
i=3
i=4
--------------------------------
Process exited after 0.1123 seconds with return value 0
Press any key to continue . . .
```

Continue Statement Output:-

```
E:\TOPS Tech Documents\Mo

i=1
i=2
i=4
--------------------------------
Process exited after 0.08628 seconds with return value 0
Press any key to continue . . .
```

14. What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

- A functions is a block of code that perform a specific tasks.
- It helps divide the big problems into smaller parts.
- Every C program has at least one function – main().

- **Function Declaration :-**
  - ✓ Tells the compiler the function is exist.
  - ✓ Specifies
  - ✓ Return type
  - ✓ Function name
  - ✓ Parameters written before main()

Syntax :- return type function_name(p1,p2,p3);

Example:- int add(int,int);

- **Function Definition :-**
  - ✓ Contain actual code of the function.
  - ✓ Written after main() or before it.

Syntax :-

```
Return type function_name(parameters){

        //code

}
```

Example :-

```
Int add(int a, int b){

        Return a + b;

}
```

- **Function call :-**
  - ✓ Used to execute the function
  - ✓ Can be called from main() or another function

Syntax:-

Function_name(arguments);

Example:-

Sum = Add(10,20);

Program Example :-

Input:-

```
    function program and example
*/

#include<stdio.h>

void add(int a, int b){
    printf("\n Addition of %d and %d is:%d ",a,b,a+b);
}

int main()
{
    add(30,40);
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

 Addition of 30 and 40 is:70
--------------------------------
Process exited after 0.05829 seconds with return value 0
Press any key to continue . . .
```

15. Write a C program that calculates the factorial of a number using a function. Include function declaration, definition, and call.

⇨ **Input:-**

```c
// Find A FACTORIAL PROGRAM USING RECURSION FUNCTION
/*
    Formula :- n!=n*(n-1)
*/

#include<stdio.h>

int factFind(int num){
    if(num==1){
        return 1;
    }
    int f;
    //formula of factorial
    f=num*factFind(num-1); //5*ff(4)
    return f;
}
main()
{
    printf("\n Factorial is = %d",factFind(7)); //function call
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

 Factorial is = 5040
--------------------------------
Process exited after 0.0575 seconds with return value 0
Press any key to continue . . .
```

16. Explain what pointers are in C and how they are declared and initialized.

    Why are pointers important in C?

⇨ **Pointer :-** pointer is a variable that stores the address of the another variable.

⇨ **Declaration of pointer :-**

Syntax :- data_type *pointer_name;

Example :- int *p; => *p is a pointer variable

⇨ **Initialization of a pointer :-**

Int a = 10;

Int *p;

p=&a;

⇨ Pointers are very important :-

- ✓ Dynamic Memory allocation
- ✓ Passing argument by reference
- ✓ Efficient array and string handling
- ✓ Building complex data structure.

- ✓ Low-level memory access.

17. Write a C program to demonstrate pointer usage. Use a pointer to modify the value of a variable and print the result. - LAB EXERCISE

⇨ **Input:-**

check-palindrome.c pointer.c

```c
/*
    Pointer Example
*/

#include<stdio.h>
int main()
{
    int a = 10;
    int *ptr = &a; //pointer declaration
    printf("\n address = %p and value = %d",ptr,*ptr); //deref

    *ptr = 200;
    printf("\n a = %d",a);

    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

 address = 000000000065FE44 and value = 10
 a = 200
--------------------------------
Process exited after 0.06172 seconds with return value 0
Press any key to continue . . .
```

18. Explain the concept of arrays in C. Differentiate between one-dimensional and multi-dimensional arrays with examples.

⇨ Array is used to store a collection of data, and it is often used as a collection variables of same data type that is known as array.

⇨ There are basically two types of array

- ➢ One Dimensional Array
- ➢ Two Dimensional Array (2-D Array)

Syntax of Array :- datatype array_name[size];

Difference between one-dimensional and multi-dimensional array :-

One Dimensional Array	Two Dimensional Array (2-D)
An array with only one index	An array with more than one index
Datatype array_name[size];	Datatype array_name[size1][size2];
Using one index access method : arr[i]	Using multiple indices access method: arr[i][j]
Stored sequentially in memory	Stored in row-major order
Using linear list structure	Using table(2D), cube(3D) etc.
Example :- int a[5];	Example:- int b[3][5];

Example :-

Input :-

Array1.c | sort-array.c | sum-of-array.c | 2D-

```c
/*
    //Array
    syntax :- datatype array_name[size];
*/

#include<stdio.h>
int main()
{
    //array declaration
    int a[5] = {12,2,34,56,12};
    int i;

    for(i=0;i<5;i++){
        printf("\n a[%d] = %d",i,a[i]);
    }
    return 0;
}
```

Output:-

E:\TOPS Tech Documents\Mo

```
 a[0] = 12
 a[1] = 2
 a[2] = 34
 a[3] = 56
 a[4] = 12
--------------------------------
Process exited after 0.05733 seconds with return value 0
Press any key to continue . . .
```

19. Write a C program that stores 5 integers in a one-dimensional array and prints them. Extend this to handle a two-dimensional array (3x3 matrix) and calculate the sum of all elements. - LAB EXERCISE

⇨ **Input:-**

Array1.c | sort-array.c | sum-of-array.c | 2D-array.c | a

```c
Part - 2 => Extend this to handle a two-dimensional array (3x3 matrix) and
calculate the sum of all elements.

*/

#include<stdio.h>
int main()
{
    int matrix[3][3],sum=0;
    int row,col;

    for(row=0;row<3;row++){
        for(col=0;col<3;col++){
            printf("Enter the arry[%d][%d] matrix value: ",row,col);
            scanf("%d", &matrix[row][col]);
        }
    }

    for(row=0;row<3;row++){
        for(col=0;col<3;col++){
            printf("\nMatrix value are: ");
            printf("%d",matrix[row][col]);
            sum = sum + matrix[row][col];
        }
        printf("\n");
    }
    printf("\n Sum of all array element: %d",sum);

    return 0;
}
```

Output:-

E:\TOPS Tech Documents\Mo

```
Enter the arry[0][0] matrix value: 10
Enter the arry[0][1] matrix value: 20
Enter the arry[0][2] matrix value: 30
Enter the arry[1][0] matrix value: 40
Enter the arry[1][1] matrix value: 50
Enter the arry[1][2] matrix value: 60
Enter the arry[2][0] matrix value: 70
Enter the arry[2][1] matrix value: 80
Enter the arry[2][2] matrix value: 90

Matrix value are: 10
Matrix value are: 20
Matrix value are: 30

Matrix value are: 40
Matrix value are: 50
Matrix value are: 60

Matrix value are: 70
Matrix value are: 80
Matrix value are: 90

 Sum of all array element: 450
--------------------------------
Process exited after 11.62 seconds with return value 0
Press any key to continue . . .
```

20. Explain string handling functions like strlen(), strcpy(), strcat(), strcmp(), and strchr(). Provide examples of when these functions are useful.

⇨ Below the string handling :-

➢ **Strlen():-** return the length of a string, not counting null character

Example :-

Input:-

```c
/*
    Find the string length using strlen() function
*/

#include<stdio.h>
#include<string.h>

int main(){
    char str[10],str1[10];

    printf("Enter the String: ");
    gets(str1);
//  puts(str1);

    printf("Length of string: %d",strlen(str1));

    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the String: Yash Tailor
Length of string: 11
--------------------------------
Process exited after 6.95 seconds with return value 0
Press any key to continue . . .
```

➢ **Strcpy():-** Copy One string to another.

Example :-

Input:-

simple-interest.c | string-length-array.c | string-strlen().c

```c
/*
    strcpy() function :- copy string to the another
    there are two main things :-
    source and destination
*/
#include<stdio.h>
#include<string.h>

int main()
{
    char str[20],copy[20];

    printf("Enter the String: ");
    gets(str);

    strcpy(copy,str);

    printf("Copy string => %s",copy);
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the String: Harsh Lakdiwala
Copy string => Harsh Lakdiwala
--------------------------------
Process exited after 6.042 seconds with return value 0
Press any key to continue . . .
```

- **Strcat() :-** Append one string to another, creating a single string out of two.

Example:-

Input:-

```c
/*
    strcat() :- Append one string to another, creating a single string out of two.
    strcat() :- source and destination
*/
#include<stdio.h>
#include<string.h>
int main()
{
    char str1[10];
    char str2[10];

    printf("Enter the string 1 => ");
    gets(str1);

    printf("\n Enter the string 2 => ");
    gets(str2);

    strcat(str1,str2);

    printf("Concatination of two strings: %s",str1);
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the string 1 => Parth

 Enter the string 2 => Gamit
Concatination of two strings: ParthGamit
--------------------------------
Process exited after 5.77 seconds with return value 0
Press any key to continue . . .
```

- **Strcmp():-** Compare two strings. If the string match , the function returns 0.

Example :-

Input:-

string-strcmp().c

```c
/*
    strcmp():- compare the two strings if strings is match then function returns 0;
    strcmp(source,destination);
*/
#include<stdio.h>
#include<string.h>
int main()
{
    char s1[20];
    char s2[20];

    printf("Enter the string 1 => ");
    gets(s1);

    printf("Enter the string 2 => ");
    gets(s2);

    if(strcmp(s1,s2)==0){
        printf("The string is equal or match");
    }else{
        printf("The string is not match");
    }
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the string 1 => yash tailor
Enter the string 2 => yash tailor
The string is equal or match
--------------------------------
Process exited after 6.096 seconds with return value 0
Press any key to continue . . .
```

- **Strchr():-** It is used to locate first occurrence of a specified character within a string.

Example:-

Input:-

```c
*/
#include<stdio.h>
#include<string.h>
int main()
{
    char str[20];
    char ch;
    char *ptr;

    printf("Enter the string: ");
    gets(str);

    printf("Enter the string you want to search a specified position: ");
    scanf("%c",&ch);

    ptr = strchr(str,ch);

    if(ptr!=NULL){
        printf("Character '%c' is found at the position: %d",ch,ptr-str);
    }else{
        printf("Character '%c' is not found at the position.",ch);
    }
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the string: Batman the world
Enter the string you want to search a specified position: e
Character 'e' is found at the position: 9
--------------------------------
Process exited after 7.863 seconds with return value 0
Press any key to continue . . .
```

21. Write a C program that takes two strings from the user and concatenates them using strcat(). Display the concatenated string and its length using strlen(). - LAB EXERCISE

⇨ **Input:-**

string-strlen().c | strcat-strlen()-prog.c

```c
/*
    Aim :- Write a C program that takes two strings from the user and concatenates them using strcat().
    Display the concatenated string and its length using strlen().
*/

#include<stdio.h>
#include<string.h>
int main()
{
    char str[20];
    char ch[20];

    printf("Enter the string 1: ");
    gets(str);

    printf("Enter the string 2: ");
    gets(ch);

    strcat(str,ch);

    printf("\nConcatenate the strings: %s",str);
    printf("\nLength of string:%d",strlen(str));
    return 0;
}
```

Output:-

```
E:\TOPS Tech Documents\Mo

Enter the string 1: Yash
Enter the string 2: Tailor

Concatenate the strings: YashTailor
Length of string:10
--------------------------------
Process exited after 4.084 seconds with return value 0
Press any key to continue . . .
```

22. Explain the concept of structures in C. Describe how to declare, initialize, and access structure members.

- A structure in C user-defined data type that allow you to group variable of different data types under a single name
- **Declaring a Structure:-**

```
Struct structure_name{
        Datatype_member1;
        Datatype_member2;
        Datatype_member3;
};
```

- **How to create structure variable :-**

```
struct Student s1, s2;
```

- **How to Initialize a Structure :-**

```
struct Student s1 = {1, "Yash", 85.5};
```

- **How to Access Structure Members :-**

Using Dot Operator (.)

```
printf("%d", s1.roll_no);
printf("%s", s1.name);
printf("%.2f", s1.marks);
```

23. Write a C program that defines a structure to store a student's details (name, roll number, and marks). Use an array of structures to store details of 3 students and print them. - LAB EXERCISE

⇨ **Input:-**

structure2.c structure-prog.c

```c
/*
    Structure LAB Execrcise

    Aim :- Write a C program that defines a structure to store a student's details (name,
           roll number, and marks). Use an array of structures to store details of 3
           students and print them.
*/
#include<stdio.h>
struct student{
    char name[20];
    int enroll;
    int marks;
};
int main()
{
    struct student s1;
    printf("\n Enter the Name, RollNo and Marks = ");
    scanf("%s %d %d",s1.name,&s1.enroll,&s1.marks);

    printf("\n Name of the student = %s",s1.name);
    printf("\n Enrollment = %d",s1.enroll);
    printf("\n Marks = %d",s1.marks);
}
```

Output:-

E:\TOPS Tech Documents\Mo

```
 Enter the Name, RollNo and Marks = Yash
101
89

 Name of the student = Yash
 Enrollment = 101
 Marks = 89
--------------------------------
Process exited after 27.06 seconds with return value 0
Press any key to continue . . .
```

24. Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.

⇨ There are below the importance of file handling :-

- **Permanent Storage** :- Data is saved even after program ends.
- **Handles Large Data** :- Store large information outside memory.
- **Data Sharing** :- Multiple programs can use the same file.
- **Record Keeping** :- useful in database, logs, reports etc.

- **Steps in File Handling :-**
  - **Declare in File pointer :-**

    FILE *fp;
  - **Open the file – fopen()**

    fp = fopen("test.txt" , "w");

    common modes :-

    "r" = read

    "w" = write

    "a" = append
  - **Write to File :-**

    fprintf(fp,"text");

    fputs("text",fp); => string write

    fputc('A' , fp); => single character passes
  - **Read From File :-**

    fscanf(fp , "%s", str);

    fgets(str, size, fp);

    fgetc(fp);
  - **Close the file :-**

    fclose(fp);

- **Example :-**
- **Input :-**

filehandling1.c filehandling-sample.c

```c
//basic example of file handling
#include<stdio.h>
int main()
{
    FILE *fp;

    fopen("hello.txt","w");
    fprintf(fp, "%s", "Hello Text");
    fclose(fp);
    return 0;
}
```

25. Write a C program to create a file, write a string into it, close the file, then open the file again to read and display its contents.

⇨ **Input :-**

```c
    Aim :- Write a C program to create a file, write a string into it, close the file, then
           open the file again to read and display its contents.
*/

#include<stdio.h>
int main()
{
    FILE *fp;
    char data[50],ch;
    fopen("sample.txt","w");

    if(fp==NULL){
        printf("Something went wrong!...");
    }
    fprintf(fp, "%s", "Welcome to our file handling process");
    fclose(fp);

    //read from the file
    fp = fopen("sample.txt","r");
    fscanf(fp, "%s",data);
    printf("\n Reading data from files = %s",data);

    //read and display file content
    while((ch=fgetc(fp))!=EOF){
        printf("%c",ch);
    }
    fclose(fp);
    return 0;
}
```

⇨ **Output:-**

```
E:\TOPS Tech Documents\Mo

--------------------------------
Process exited after 2.348 seconds with return value 3221225477
Press any key to continue . . .
```

⇨ **File created sample.txt**